

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
Harding Rafif Dzakwan Permana
10908230018
S1 IF-12-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Mesin abstrak merupakan model komputasi yang dibuat di atas mesin komputasi lain dengan menyediakan tipe data dan operasi-operasi dasar tertentu untuk menyelesaikan suatu permasalahan. Mesin abstrak digunakan agar proses pembuatan program menjadi lebih terstruktur dan mudah dipahami karena setiap operasi dibuat sesuai kebutuhan data yang digunakan. Pada Modul 18, mesin abstrak diterapkan pada permainan domino dengan menyediakan operasi dasar seperti mengocok kartu, mengambil kartu, melihat gambar kartu, dan menghitung nilai kartu domino. Dengan adanya operasi-operasi tersebut, program dapat mensimulasikan berbagai permainan domino secara lebih sederhana dan terorganisir.

Selain mesin domino, modul ini juga membahas mesin abstrak karakter yang bekerja terhadap untaian karakter hingga menemukan tanda akhir berupa titik (.). Mesin karakter memiliki operasi dasar seperti `start()` untuk memulai pembacaan karakter, `maju()` untuk berpindah ke karakter berikutnya, `eop()` untuk mendeteksi akhir pembacaan, dan `cc()` untuk mengambil karakter yang sedang dibaca. Dengan operasi dasar tersebut, program dapat melakukan berbagai proses seperti membaca seluruh karakter, menghitung jumlah karakter, menghitung jumlah huruf tertentu, mencari frekuensi kemunculan huruf, hingga mendeteksi pasangan karakter tertentu pada sebuah kalimat.

Latihan Soal

1. Implementasi Operasi Dasar Mesin Domino

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu [28]Domino
    jumlah int
}

func buatDomino() Dominoes {

    var d Dominoes
    index := 0

    for i := 0; i <= 6; i++ {

        for j := i; j <= 6; j++ {

            d.kartu[index].sisi1 = i
            d.kartu[index].sisi2 = j

            d.kartu[index].nilai = i + j

            if i == j {
                d.kartu[index].balak = true
            } else {
                d.kartu[index].balak = false
            }

            index++
        }
    }

    d.jumlah = 28
}
```

```

    return d
}

func kocokKartu(d *Dominoes) {

    rand.Seed(time.Now().UnixNano())

    for i := 0; i < d.jumlah; i++ {

        j := rand.Intn(d.jumlah)

        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {

    kartu := d.kartu[d.jumlah-1]

    d.jumlah--

    return kartu
}

func gambarKartu(d Domino, suit int) int {

    if suit == 1 {
        return d.sisi1
    }

    return d.sisi2
}

func nilaiKartu(d Domino) int {
    return d.nilai
}

func main() {

    domino := buatDomino()

    kocokKartu(&domino)

    kartu := ambilKartu(&domino)

    fmt.Println("Kartu yang diambil:")
    fmt.Println(kartu)

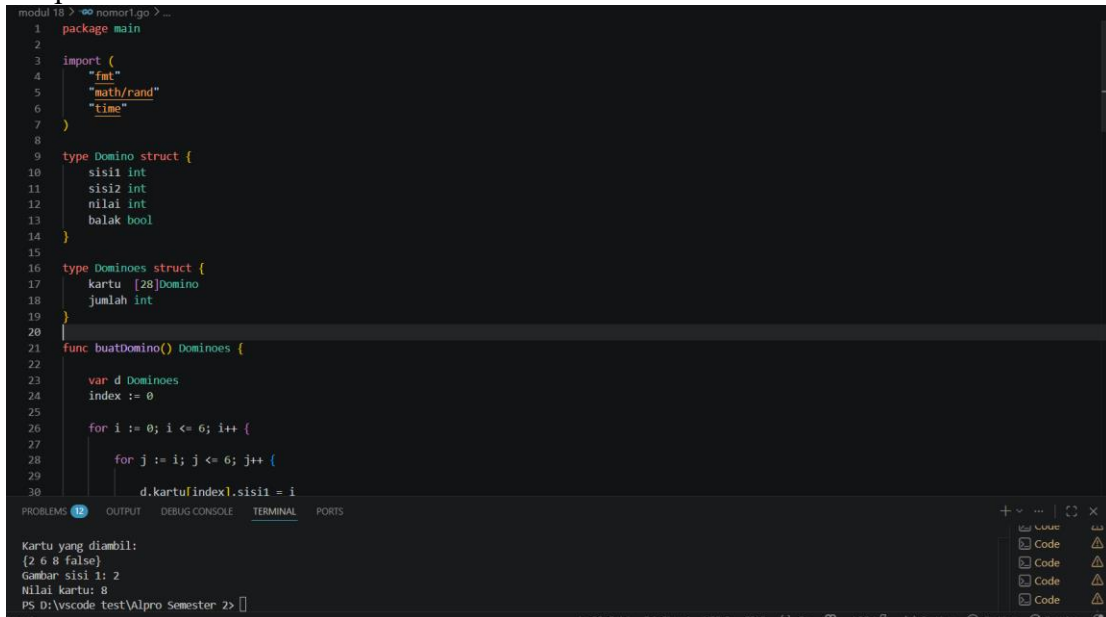
    fmt.Println("Gambar sisi 1:", gambarKartu(kartu, 1))

    fmt.Println("Nilai kartu:", nilaiKartu(kartu))
}

```

```
}
```

Output:



```
modul 18 > cd nomor1.go > ...
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 type Domino struct {
10     sisi1 int
11     sisi2 int
12     nilai int
13     balak bool
14 }
15
16 type Dominoes struct {
17     kartu [28]Domino
18     jumlah int
19 }
20
21 func buatDomino() Dominoes {
22
23     var d Dominoes
24     index := 0
25
26     for i := 0; i <= 6; i++ {
27         for j := i; j <= 6; j++ {
28             d.kartu[index].sisi1 = i
29             d.kartu[index].sisi2 = j
30             index++
31         }
32     }
33
34     return d
35 }
36
37 func main() {
38     d := buatDomino()
39     fmt.Println("Kartu yang diambil:", d.kartu[0])
40     fmt.Println("Gambar sisi 1:", d.kartu[0].sisi1)
41     fmt.Println("Nilai kartu:", d.kartu[0].nilai)
42 }
```

Kartu yang diambil:
{2 6 8 false}
Gambar sisi 1: 2
Nilai kartu: 8
PS D:\vscode test\Alpro Semester 2>

Deskripsi Program:

Program ini dibuat untuk mensimulasikan mesin abstrak domino. Program dapat membuat satu set kartu domino, mengocok kartu secara acak, mengambil kartu dari tumpukan, melihat gambar/sisi kartu, dan menghitung nilai kartu domino. Program ini menggunakan struct dan fungsi agar operasi domino lebih terstruktur dan mudah digunakan.

type Domino struct { **menyimpan data 1 kartu domino**

sisi1 int **angka sisi pertama**

sisi2 int **angka sisi kedua**

nilai int **jumlah kedua sisi**

balak bool **true jika kedua sisi sama**

}

type Dominoes struct { **Digunakan untuk menyimpan seluruh kartu domino, jumlah kartu yang masih tersedia**

kartu [28]Domino **standar kartu**

jumlah int

}

Digunakan agar kartu tidak duplikat.

```
for i := 0; i <= 6; i++ {  
    for j := i; j <= 6; j++ {
```

Mengecek Balak, Jika kedua sisi sama maka kartu disebut balak

```
if i == j {  
    d.kartu[index].balak = true  
}
```

func kocokKartu(d *Dominoes) mengacak urutan kartu domino.

```
d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i] Menukar  
posisi kartu agar susunan menjadi acak.
```

**func ambilKartu(d *Dominoes) Domino Digunakan untuk
mengambil 1 kartu dari tumpukan.**

```
kartu := d.kartu[d.jumlah-1] Mengambil kartu terakhir.
```

```
d.jumlah-- Jumlah kartu dikurangi karena sudah diambil.
```

**func nilaiKartu(d Domino) int { Digunakan untuk mengambil
total nilai kartu.**

```
    return d.nilai
```

```
}
```

Urutan proses :

1. Membuat kartu domino
2. Mengocok kartu
3. Mengambil kartu
4. Menampilkan hasil

2. Realisasi Aksi Mesin Domino

A. Source Code:

```
func galiKartu(d *Dominoes, kartu Domino) {  
  
    var ambil Domino  
    ketemu := false  
  
    for d.jumlah > 0 && !ketemu {  
  
        ambil = ambilKartu(d)  
  
        if ambil.sisi1 == kartu.sisi1 ||  
           ambil.sisi2 == kartu.sisi1 ||  
           ambil.sisi1 == kartu.sisi2 ||  
           ambil.sisi2 == kartu.sisi2 {  
  
            fmt.Println("Kartu cocok ditemukan:")  
            fmt.Println(ambil)  
  
            ketemu = true  
        }  
    }  
  
    if !ketemu {  
        fmt.Println("Tidak ada kartu yang cocok")  
    }  
}
```

Output:

```
modul 18 > nomor2a.go  
1 func galiKartu(d *Dominoes, kartu Domino) {  
2  
3     var ambil Domino  
4     ketemu := false  
5  
6     for d.jumlah > 0 && !ketemu {  
7  
8         ambil = ambilKartu(d)  
9  
10        if ambil.sisi1 == kartu.sisi1 ||  
11           ambil.sisi2 == kartu.sisi1 ||  
12           ambil.sisi1 == kartu.sisi2 ||  
13           ambil.sisi2 == kartu.sisi2 {  
14  
15            fmt.Println("Kartu cocok ditemukan:")  
16            fmt.Println(ambil)  
17  
18            ketemu = true  
19        }  
20    }  
21  
22    if !ketemu {  
23        fmt.Println("Tidak ada kartu yang cocok")  
24    }  
25 }
```

Deskripsi Program:

- Fungsi ini mengambil kartu dari tumpukan domino satu per satu.
- Program akan berhenti jika ditemukan kartu yang memiliki gambar/suit yang sama.
- Jika tidak ada kartu yang cocok maka muncul pesan "Tidak ada kartu yang cocok".

2B. Source Code:

```
func sepasangKartu(a Domino, b Domino) bool {  
    total := nilaiKartu(a) + nilaiKartu(b)  
    return total == 12  
}
```

Output:

```
func sepasangKartu(a Domino, b Domino) bool {  
    total := nilaiKartu(a) + nilaiKartu(b)  
    return total == 12  
}
```

Deskripsi Program:

- Fungsi ini digunakan untuk mengecek apakah total nilai dua kartu domino = 12.
- Jika total 12 maka menghasilkan true.
- Jika bukan 12 maka menghasilkan false.

3. Implementasi salah satu permainan domino

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    sisi1 int
    sisi2 int
}

type Dominoes struct {
    kartu [28]Domino
    jumlah int
}

func buatDomino() Dominoes {

    var d Dominoes
    index := 0

    for i := 0; i <= 6; i++ {

        for j := i; j <= 6; j++ {

            d.kartu[index].sisi1 = i
            d.kartu[index].sisi2 = j

            index++
        }
    }

    d.jumlah = 28

    return d
}

func kocokKartu(d *Dominoes) {

    rand.Seed(time.Now().UnixNano())

    for i := 0; i < d.jumlah; i++ {

        j := rand.Intn(d.jumlah)

        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
```

```

    kartu := d.kartu[d.jumlah-1]

    d.jumlah--

    return kartu
}
func bisaMain(a Domino, b Domino) bool {

    return a.sisi1 == b.sisi1 ||
           a.sisi1 == b.sisi2 ||
           a.sisi2 == b.sisi1 ||
           a.sisi2 == b.sisi2
}
func main() {

    domino := buatDomino()

    kocokKartu(&domino)

    k1 := ambilKartu(&domino)
    k2 := ambilKartu(&domino)

    fmt.Println("Kartu pertama :", k1)
    fmt.Println("Kartu kedua   :", k2)

    if bisaMain(k1, k2) {

        fmt.Println("Kartu bisa disambungkan")

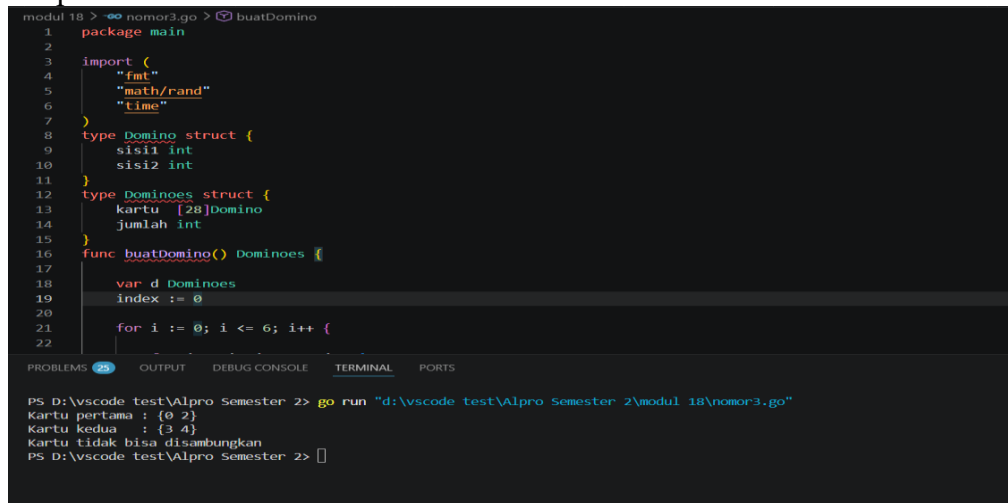
    } else {

        fmt.Println("Kartu tidak bisa disambungkan")

    }
}

```

Output:



The screenshot shows a Go IDE with the source code on the left and the terminal output on the right. The source code is the same as the one in the previous block. The terminal output shows the execution of the program, which prints the first and second cards and then checks if they can be connected. In this run, the cards are {0 2} and {3 4}, and the output is "Kartu tidak bisa disambungkan".

```

modul 18 > nomor3.go > buatDomino
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8 type Domino struct {
9     sisi1 int
10    sisi2 int
11 }
12 type Dominoes struct {
13     kartu [28]Domino
14     jumlah int
15 }
16 func buatDomino() Dominoes {
17
18     var d Dominoes
19     index := 0
20
21     for i := 0; i <= 6; i++ {
22

```

```

PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 18\nomor3.go"
Kartu pertama : {0 2}
Kartu kedua   : {3 4}
Kartu tidak bisa disambungkan
PS D:\vscode test\Alpro Semester 2>

```

Deskripsi Program:

program mengambil 2 kartu domino
lalu mengecek apakah kedua kartu bisa dimainkan pada
permainan Gapleh
syaratnya harus ada sisi angka yang sama

4. Implementasi Mesin Abstrak Karakter

Source Code:

```
package main

import "fmt"

var pita string
var indeks int
var currentChar byte

func start() {
    indeks = 0
    currentChar = pita[indeks]
}

func maju() {
    indeks++

    if indeks < len(pita) {
        currentChar = pita[indeks]
    }
}

func eop() bool {
    return currentChar == '.'
}

func cc() byte {
    return currentChar
}

func main() {
    fmt.Print("Masukkan kalimat diakhiri titik : ")
    fmt.Scan(&pita)

    start()
}
```

```
jumlahKarakter := 0
jumlahA := 0
jumlahLE := 0

var prev byte

for !eop() {

    fmt.Print(string(cc()))

    jumlahKarakter++

    if cc() == 'A' {
        jumlahA++
    }

    if prev == 'L' && cc() == 'E' {
        jumlahLE++
    }

    prev = cc()

    maju()
}

fmt.Println()
fmt.Println("Jumlah karakter :", jumlahKarakter)

fmt.Println("Jumlah huruf A :", jumlahA)

frekuensi := float64(jumlahA) /
float64(jumlahKarakter)

fmt.Println("Frekuensi huruf A :", frekuensi)

fmt.Println("Jumlah pasangan LE :", jumlahLE)
}
```

Output:

```
modul 18 > -o nomor4.go > main
1 package main
2
3 import "fmt"
4
5 var pita string
6 var indeks int
7 var currentChar byte
8
9 func start() {
10     indeks = 0
11     currentChar = pita[indeks]
12 }
13
14 func maju() {
15     indeks++
16
17     if indeks < len(pita) {
18         currentChar = pita[indeks]
19     }
20 }
21
22 func eop() bool {
23     return currentChar == '.'
24 }
25
26
27
28
```

PROBLEMS 27 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 18\nomor4.go"
Masukkan kalimat diakhiri titik : ALELENA.
ALELENA
Jumlah karakter : 7
Jumlah huruf A : 2
Frekuensi huruf A : 0.2857142857142857
Jumlah pasangan LE : 2
PS D:\vscode test\Alpro Semester 2> []
```

Deskripsi Program:

Program ini merupakan implementasi mesin abstrak karakter yang digunakan untuk membaca sebuah untaian karakter hingga menemukan tanda titik (.) sebagai akhir kalimat

Membaca karakter satu per satu sampai tanda titik (.).

Menggunakan operasi dasar mesin karakter:

- start()
- maju()
- eop()
- cc()

Menghitung jumlah seluruh karakter.

Menghitung jumlah huruf A.

Menghitung frekuensi huruf A.

Menghitung jumlah pasangan huruf "LE".

Menampilkan hasil perhitungan ke layar

DAFTAR PUSTAKA

<https://www.youtube.com/playlist?list=PLjRBWix725xrVtr7q2Asm5LdTmTNSbH8A>